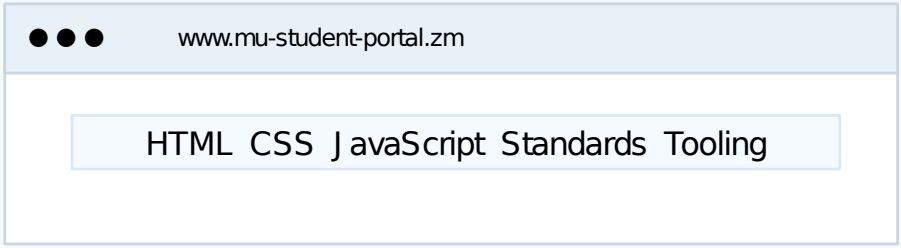
MULUNGUSHI UNIVERSITY
School of Engineering and Technology

ICT461

Web Systems and Technology

Unit 1: Web Platform and Standards

Practical, competency based, industry oriented slides for 4th year Computer Science students



Mr E Nyirenda

Why web systems matter now

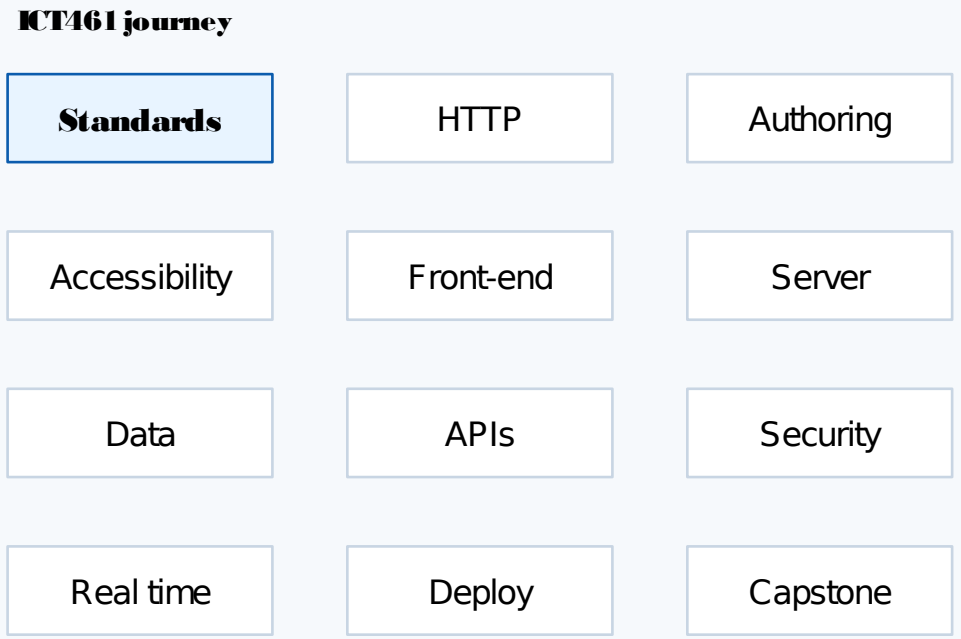
Every bank, university, hospital and shop now depends on web platforms.
The browser is both a user interface and an execution environment.
Industry expects graduates who can build, test, secure and deploy working systems.
This unit sets the professional foundation: standards, browsers and tooling.

Industry connection

A student portal failure during registration is not only a programming problem. It is a standards, browser, network, accessibility, performance and security problem working together.

Course map: where Unit 1 fits

Unit 1 gives the language of the modern Web. Later units build on it: HTTP, authoring, accessibility, front-end programming, APIs, data, security, real time, media and deployment. Capstone expectation: a production style web system with evidence of quality.



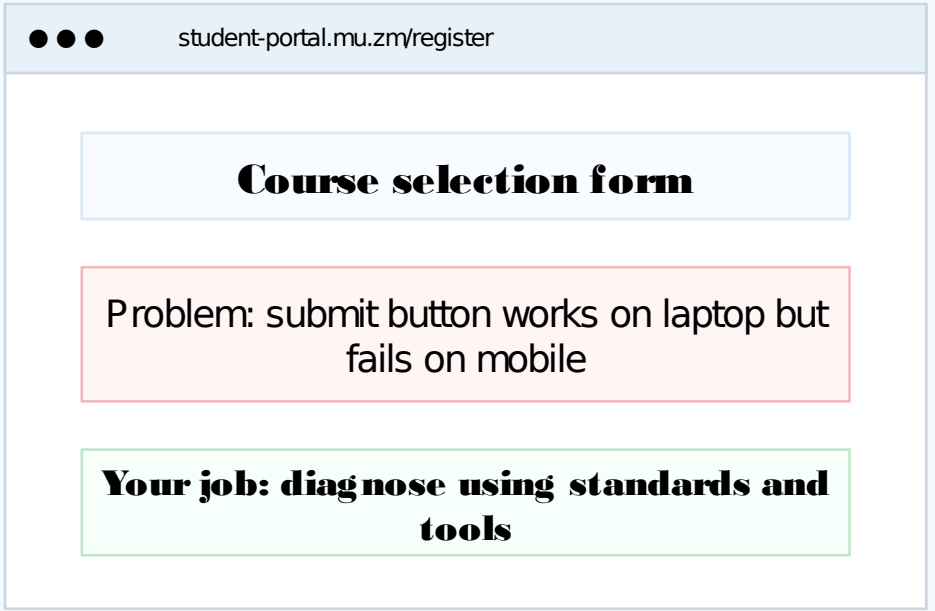
Competencies for this lecture

Explain the role of Web standards in professional development.
Describe the browser as a platform, not just an app.
Use developer tools to inspect a real web page.
Create a small standards compliant page and justify design choices.
Work like an industry team: Git, issues, review and demo.

By the end, you should be able to explain, inspect, build and defend a basic standards based web page.

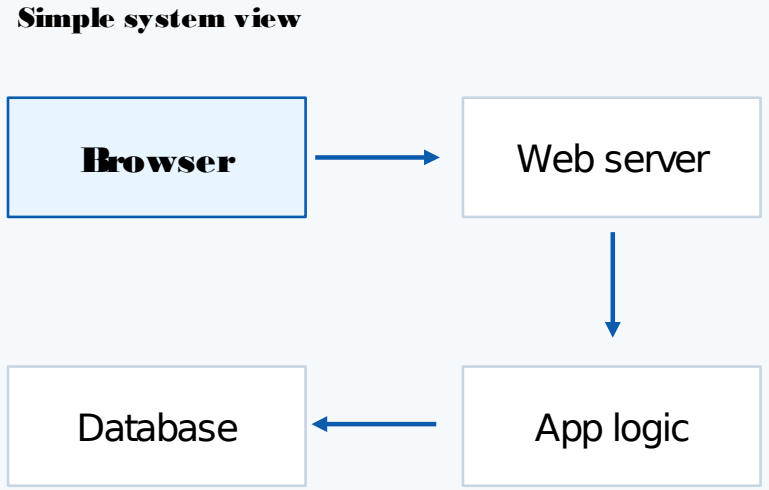
Opening scenario: registration week

A university portal slows down, forms fail on some phones, and visually impaired students cannot submit courses.
The team argues: browser issue, server issue, database issue or network issue?
A professional web engineer investigates the whole system using evidence.



What is a web system?

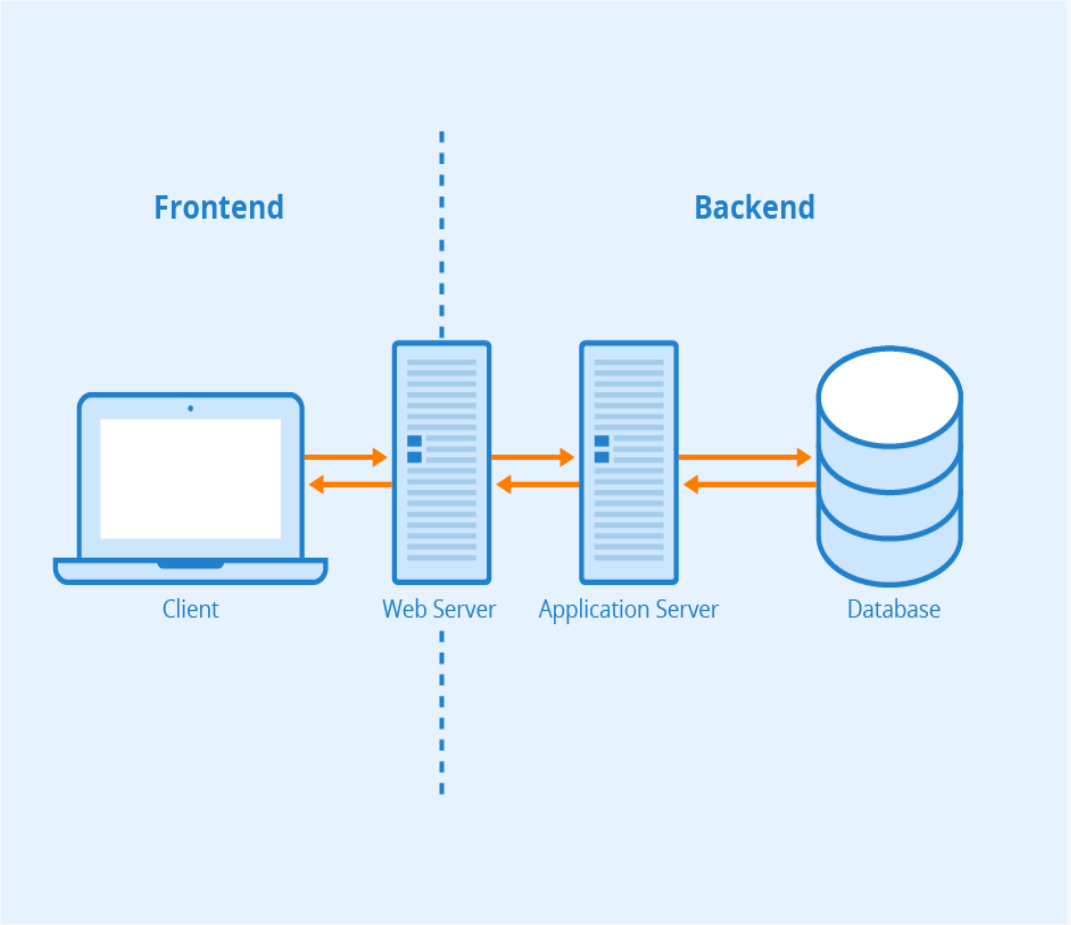
A web system is a set of cooperating parts accessed through web protocols and browsers. It includes user interface, application logic, APIs, data stores, security controls and operations. A website can be static. A web system usually changes state, integrates data and supports workflows.



Standards make the parts speak clearly.

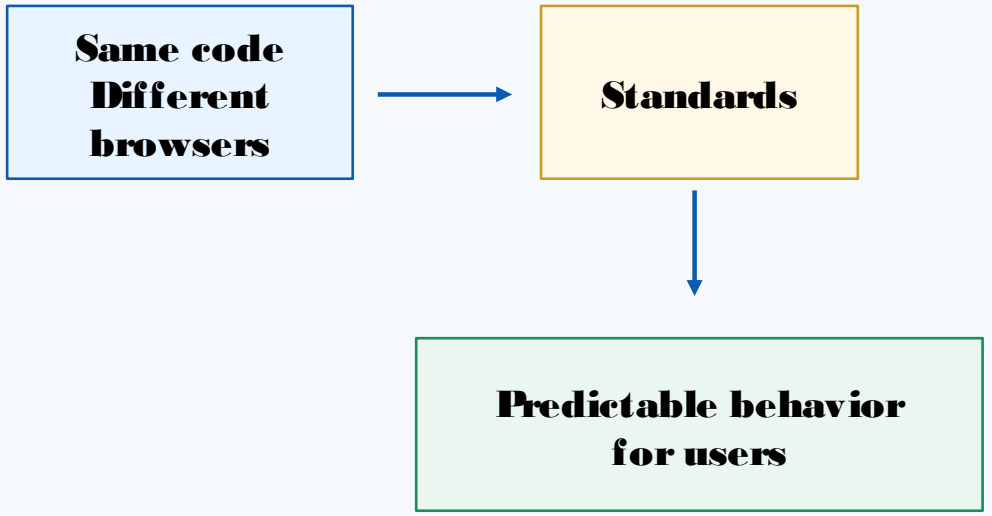
Anatomy of a modern web application

Client sends requests and receives HTML, JSON, media and scripts.
Web server handles delivery, routing and sometimes security policies.
Application server enforces business rules.
Database stores reliable and searchable state.



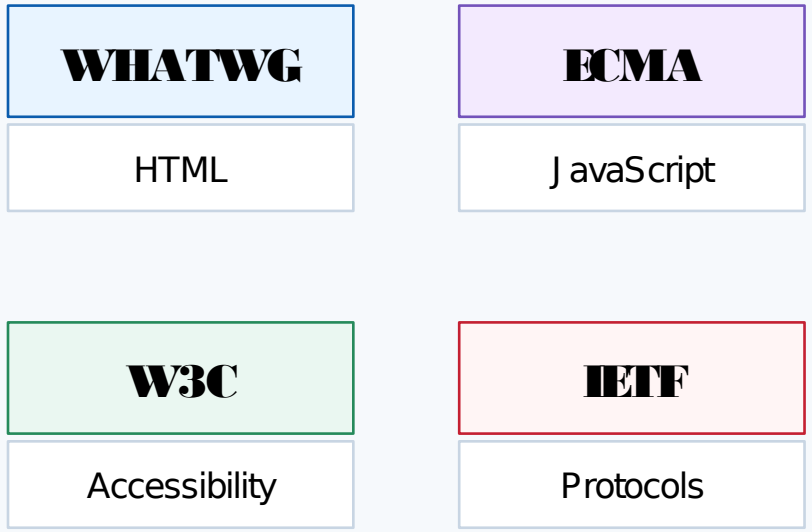
Standards: the contract of the Web

A standard is an agreed way of doing something so independent systems can interoperate.
The Web works because browsers, servers and developers follow shared contracts.
Ignoring standards often creates hidden technical debt: broken pages, accessibility failures and security gaps.



Who makes web standards?

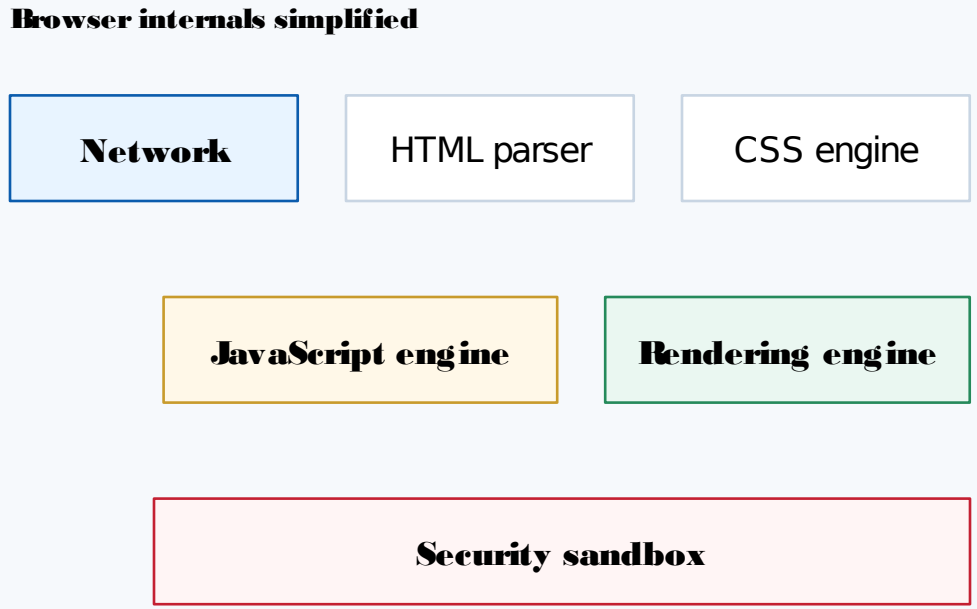
WHATWG: HTML living standard and browser platform evolution.
ECMA International: ECMAScript language specification.
W3C: accessibility, WebRTC and many Web recommendations.
IETF: HTTP, TLS, WebSocket and Internet protocols.



Professional developers read standards through practical docs, then verify behavior in the browser.

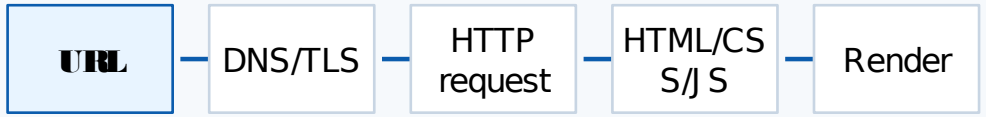
The browser is a platform

It parses HTML, CSS and JavaScript.
It renders pixels and handles input.
It exposes Web APIs such as Storage, Fetch, History and Media.
It enforces security boundaries such as origin and permissions.



From URL to rendered page

User enters URL or clicks a link.
Browser resolves address, connects securely and requests resources.
Server responds with HTML and linked assets.
Browser builds DOM, CSSOM, layout, paint and composite layers.



Every step can affect speed, security and user experience.

HTML living standard in practice

HTML describes document structure and meaning.
Modern HTML evolves continuously through the living standard model.
Browser support must still be checked because users run different versions.
Semantic elements improve accessibility, search and maintenance.

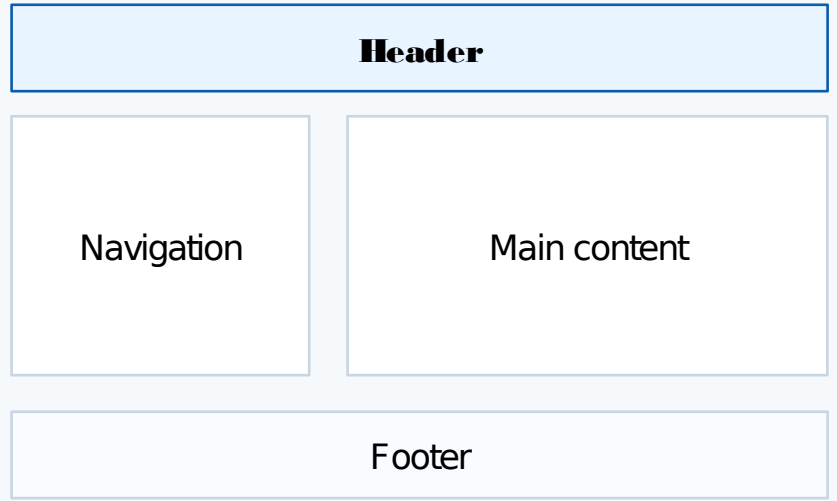
```
<main>
  <article>
    <h1>Registration Guide</h1>
    <p>Choose courses before
Friday.</p>
  </article>
</main>
```

Meaning before appearance

Semantic HTML: think like assistive technology

Use headings in logical order.
Use nav for navigation, main for main content and footer for closing information.
Use button for actions and for navigation.
Use labels with inputs. Placeholder text is not a label.

Screen reader mental map



Good structure helps all users.

Example: poor markup versus professional markup

Poor markup uses div for everything and hides meaning.
Professional markup exposes document structure.
Industry code review should check semantic correctness before styling polish.

```
Poor
<div onclick="save()">Save</div>

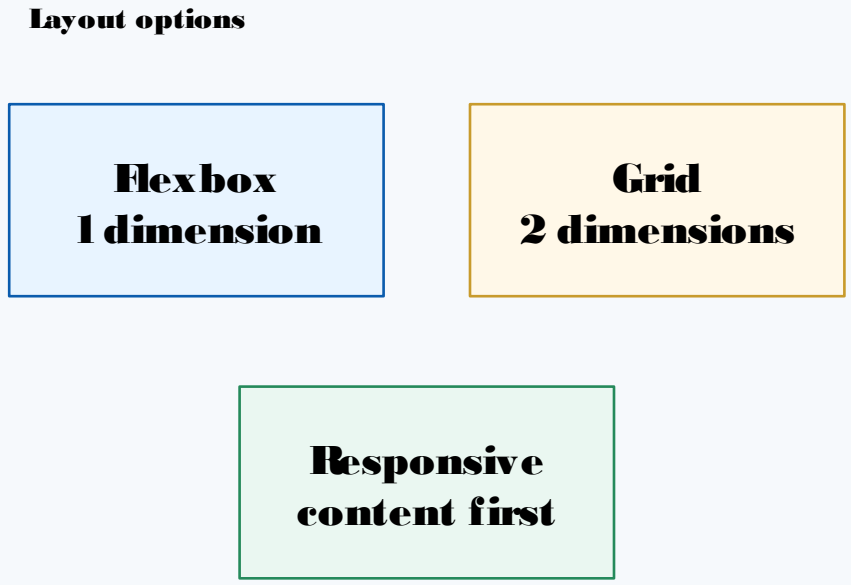
Better
<button type="submit">Save</button>

Poor
<div class="title">Fees</div>

Better
<h2>Fees</h2>
```

Modern CSS: layout as engineering

CSS controls presentation, layout, motion and responsive behavior.
Modern layouts commonly use Flexbox and Grid.
Good CSS is maintainable: naming, tokens, reusable patterns and clear breakpoints.
Design must respond to content, device size and user preferences.



Responsive design: build for real devices

A site must work on small phones, tablets, laptops and projectors.
Use responsive units, media queries and responsive images.
Test with real content and slow network profiles.
Industry principle: mobile defects are production defects.



Same task, different constraints

JavaScript and TypeScript in web systems

JavaScript adds behavior and client side logic.
TypeScript adds static typing that helps large teams reduce mistakes.
Use JavaScript for interaction, not to replace good HTML.
Keep critical validation on the server even when client validation improves UX.

```
type Course = {
  code: string;
  title: string;
  credits: number;
};

function display(course: Course) {
  return `${course.code}: ${course.title}`;
}
```

Large teams value predictable code.

ECMAScript modules: code with boundaries

Modules let teams split code into focused files.
Imports and exports make dependencies explicit.
A clean module boundary improves testing and review.
Avoid global variables in professional projects.

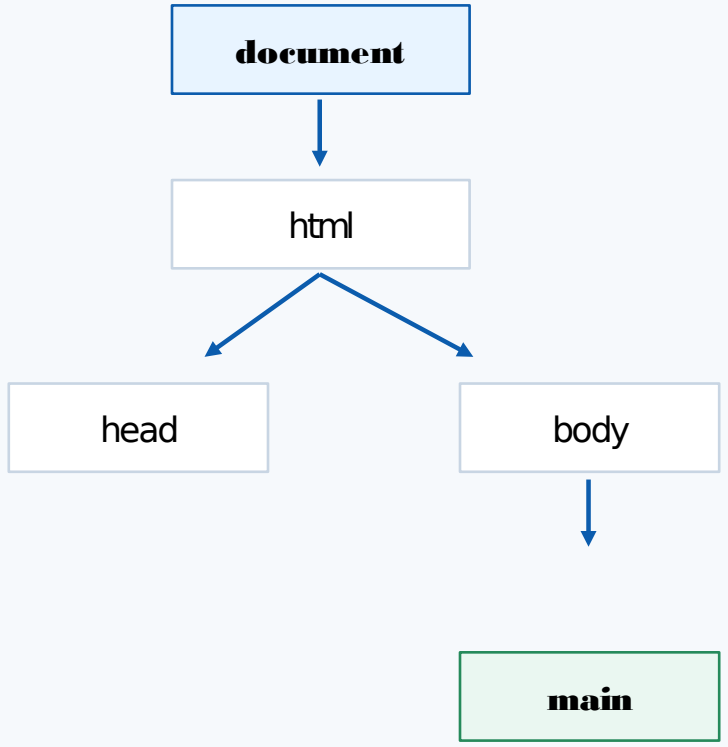
```
// fees.js
export function calculateFee(units) {
  return units * 250;
}

// app.js
import { calculateFee } from "./fees.js";

console.log(calculateFee(4));
```

DOM: the document as a programmable tree

The DOM represents HTML as objects that JavaScript can inspect and change.
DOM changes can trigger layout and paint work.
Overusing DOM updates can make applications feel slow.
Prefer clear state flow and measured updates.



Events: users drive the system

Click, input, submit, keydown and load events turn user actions into code paths.
Professional front-end code treats events as workflows.
Prevent default behavior only when necessary and understood.
Always support keyboard and assistive technology interaction.

```
const form = document.querySelector("form");

form.addEventListener("submit", async (event)
=> {
  event.preventDefault();
  // collect data
  // validate client-side
  // send to server
});
```

A form submit is a business transaction.

Fetch API: connecting front end to services

Fetch allows the browser to request data without a full page reload.
Handle success, errors, timeouts and empty states.
Never trust the client just because the browser sent the request.
Document API expectations for the team.

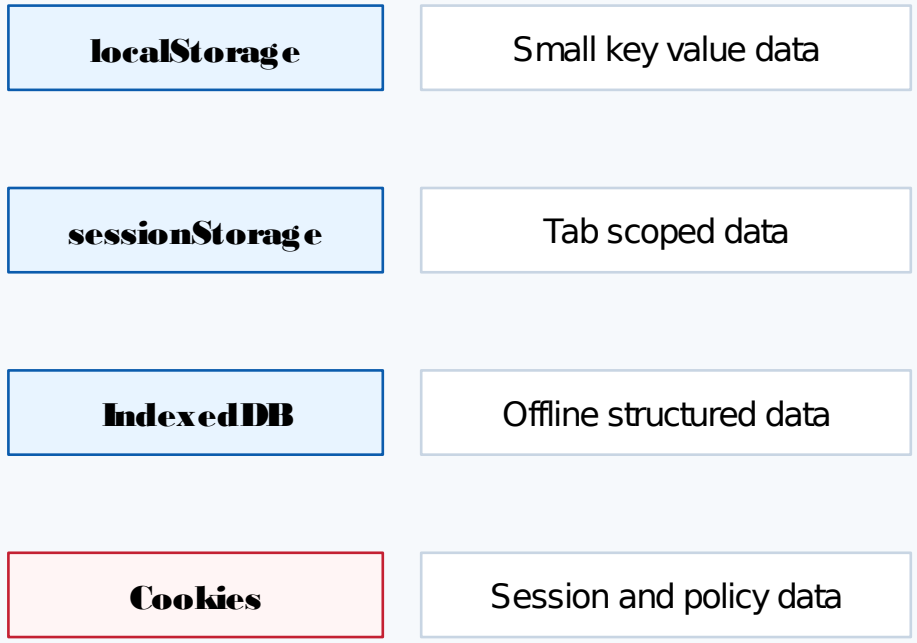
```
async function loadCourses() {
  const res = await fetch("/api/courses");

  if (!res.ok) {
    throw new Error("Could not load courses");
  }

  return await res.json();
}
```

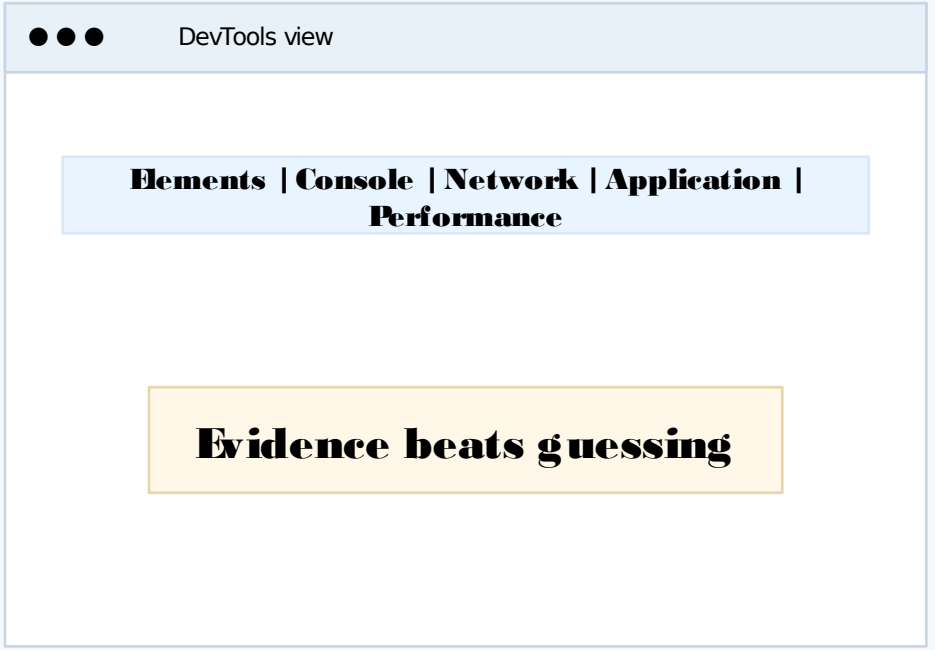
Browser storage: useful but risky

LocalStorage is simple but not suitable for sensitive tokens.
SessionStorage lasts for a tab session.
IndexedDB supports larger structured client side data.
Cookies can be protected with flags and are central to sessions.



Developer tools: your industry microscope

Elements panel: inspect HTML, CSS and layout.
Console: view errors, warnings and quick experiments.
Network: inspect requests, responses, headers and timing.
Application: inspect storage, cookies and service workers.
Performance: profile rendering and scripting cost.



Practical: inspect a live page in 5 minutes

Open a page you use often.
Inspect a heading, button and form field.
Check whether images have alternative text.
Open Network and reload the page.
Find the slowest request and explain what it is.

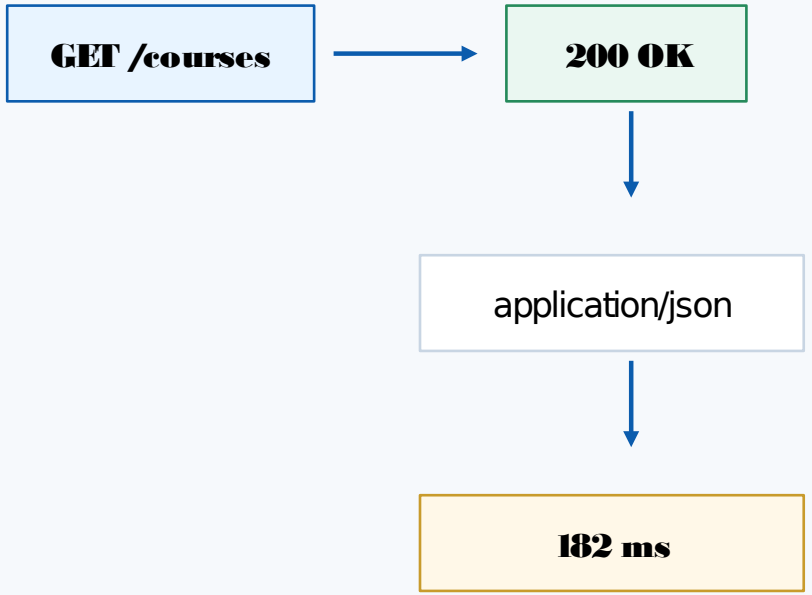
Classroom workflow



Submit a screenshot and one paragraph diagnosis.

Network tab: read the conversation

Method tells the action: GET, POST, PUT, PATCH or DELETE.
Status code tells the result category.
Headers carry metadata such as content type, cache and cookies.
Timing shows where users wait.



Performance basics: users feel delay

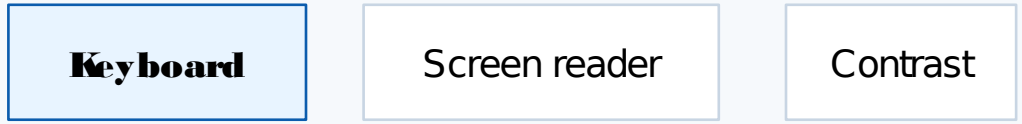
Performance is a user experience requirement, not decoration.
Large assets, blocking scripts and slow APIs reduce trust.
Core Web Vitals help teams discuss loading, responsiveness and visual stability.
Measure before optimizing.



Budget example: initial page under 250 KB compressed for low bandwidth users.

Accessibility is a professional requirement

Accessible systems can be used by people with diverse abilities and contexts.
Keyboard support, labels, contrast and meaningful structure are basic engineering tasks.
Accessibility improves quality for everyone, including mobile and low bandwidth users.
Test with tools and human judgement.



Can every student complete the same task independently?

ARIA: helpful when HTML is not enough

First rule: use native HTML elements when possible.
ARIA can add roles, names and states for assistive technologies.
Wrong ARIA can make a page worse.
Test interactive components with keyboard and screen reader workflows.

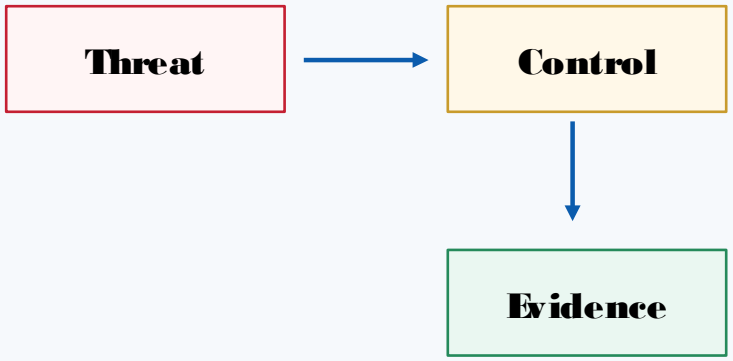
```
<button aria-expanded="false"
        aria-controls="course-menu">
  Select course
</button>

<ul id="course-menu" hidden>
  <li>ICT461</li>
</ul>
```

Use ARIA to clarify, not to repair careless HTML.

Security starts in Unit 1

Standards shape security: origins, permissions, cookies, TLS and CSP.
Client code is visible to users and attackers.
Validate on the server, encode output and protect secrets.
A professional developer asks: what can go wrong and how do we prove it is controlled?



Security is a design habit, not a final week activity.

Mini lab: standards compliant form

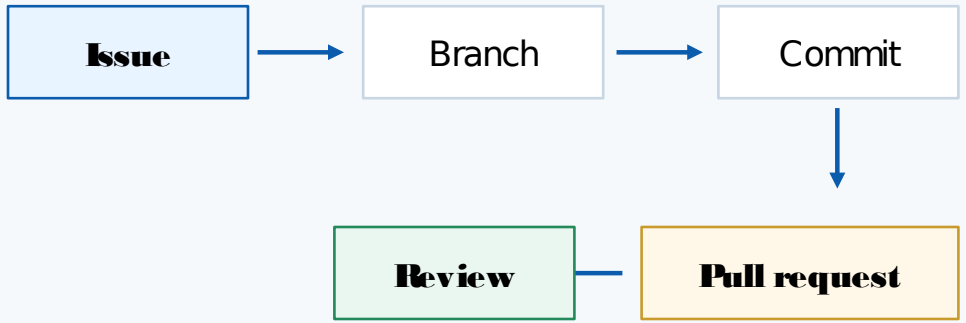
Create a course registration form with name, student ID, programme, course and submit button.
Use labels, required fields and correct input types.
Add simple client side validation messages.
Prepare to explain what must also be validated on the server.

```
<form method="post" action="/register">
  <label for="studentId">Student ID</label>
  <input id="studentId" name="studentId"
required>

  <button type="submit">Register</button>
</form>
```

Git workflow for web systems

Use Git to show how the system evolved.
Branch for each feature or fix.
Write clear commits that explain intent.
Use pull requests for review and quality gates.
Never submit only a zip file as evidence of professional work.



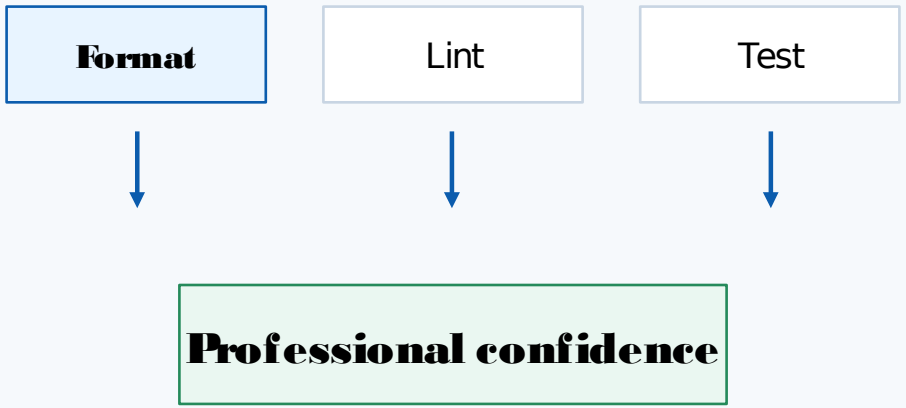
Toolchain used in this course

VS Code or equivalent editor with useful extensions.
Browser DevTools for inspection and debugging.
Node.js for tooling and local development.
Git and GitHub for version control and collaboration.
Linting and formatting to reduce avoidable defects.

Editor	VS Code
Runtime	Node.js
Versioning	Git
Browser	DevTools
Quality	Lint/Format

Quality habits from the first file

Use meaningful names and consistent formatting.
Avoid dead code and unexplained workarounds.
Create small functions with clear responsibility.
Run checks before pushing.
Review accessibility, validation and performance early.



Quality is cheaper when built in early.

Documentation: help the next developer

README explains how to run, test and understand the project.
Comments explain why, not obvious what.
Architecture Decision Records document important choices.
API documentation prevents integration guesswork.
Screenshots and demo notes make assessment evidence stronger.

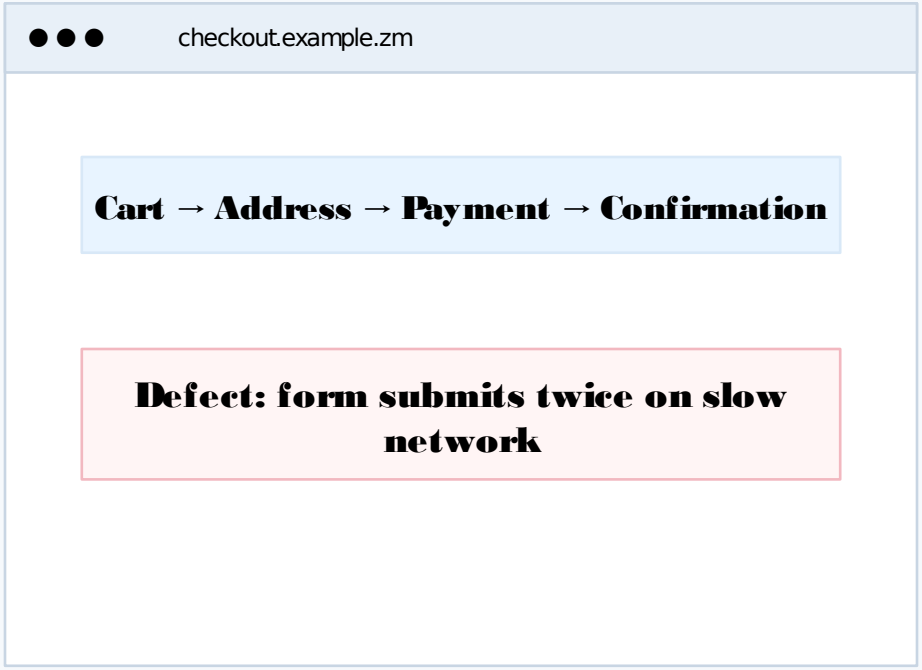
```
# Student Portal Demo

## Run
npm install
npm run dev

## Features
- Accessible registration form
- Client and server validation
- Network inspection evidence
```

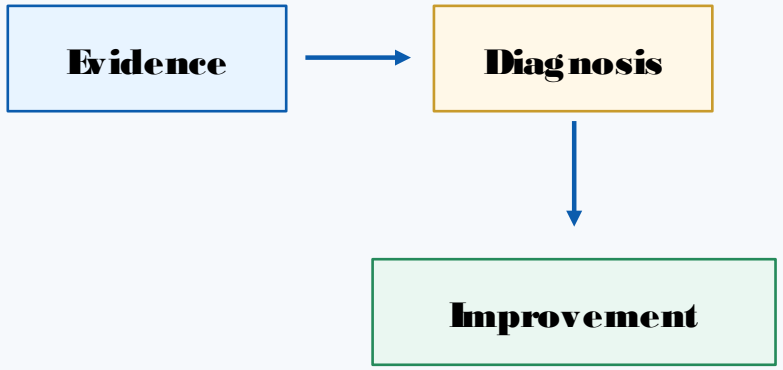
Industry case: e-commerce checkout

Checkout combines forms, validation, security, performance, accessibility and data integration.
Small errors create abandoned carts and support calls.
Standards reduce friction across browsers and devices.
The same thinking applies to student portals, hospital systems and tax systems.



Team activity: standards audit

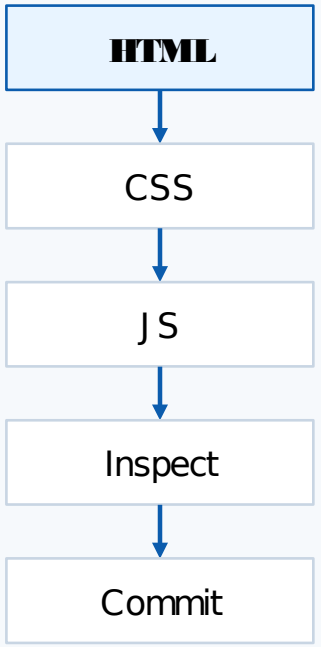
Groups choose one real website: university, bank, shop, NGO or public service.
Audit five areas: semantic structure, responsiveness, accessibility, network behavior and visible errors.
Use screenshots as evidence.
Present one risk and one improvement in three minutes.



No unsupported opinions. Show proof.

Practical demo sequence

Create index.html with semantic sections.
Add responsive CSS for mobile and desktop.
Add one JavaScript interaction with an event listener.
Inspect the page with DevTools.
Commit and push to GitHub with a useful README.



Deliverable: working page + screenshots + Git link

Competency checklist

I can explain why standards protect users and developers.
I can describe the browser rendering path in simple terms.
I can inspect HTML, CSS, network requests and storage using DevTools.
I can build a small semantic and responsive page.
I can present evidence of quality using screenshots and Git history.

Ready means you can demonstrate, not only define.

Competence = knowledge + product + evidence + reflection

Quick knowledge check

Why is button better than div for a clickable form action?
Name two standards bodies and one technology associated with each.
What evidence can the Network tab provide during debugging?
Why is client side validation not enough for security?
What would you check first when a page fails only on mobile?

Discuss in pairs for 5 minutes. Then answer with one practical example.

Unit 1 lab assignment

Build a one page course registration interface for ICT461.
Use semantic HTML, responsive CSS and one JavaScript interaction.
Include labels, keyboard friendly controls and clear validation messages.
Use DevTools to capture one Elements screenshot and one Network screenshot.
Submit GitHub link, screenshots and a 300 word reflection.

Assessment evidence

Working page
Git history
Accessibility checks
DevTools screenshots
Short reflection

Next lecture: HTTP fundamentals